

RAG AWS Documentation

Introduction

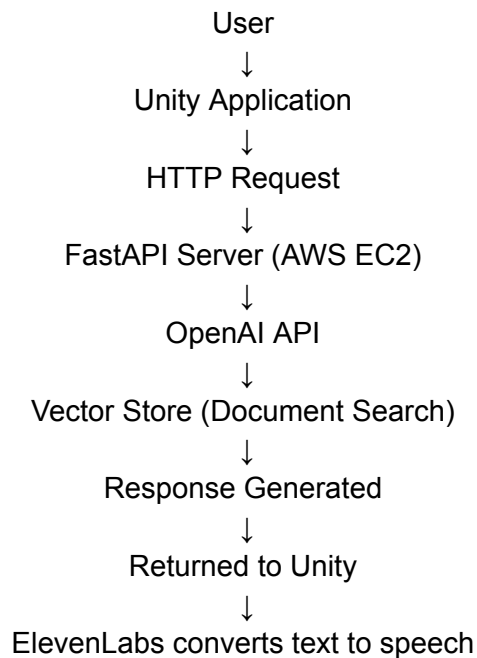
This project utilizes a cloud based Retrieval-Augmented Generation (RAG) AI server that is used on the backend of the Unity application. This allows for the user to speak naturally and ask questions to the AI knowledge base which will gather relevant information before responding to the user.

The backend server was designed to:

- Provide a reliable API endpoint for Unity
- Allow the AI to utilize project specific knowledge base
- Maintain API key security
- Run continuously on AWS servers

System Architecture

The system runs on Unity for the user and reaches the RAG server on the back end.



This design allows for easy debugging and maintenance by having it broken down into multiple parts and connecting each of them.

Cloud Infrastructure Setup

The backend server is through Amazon Web Services (AWS) Elastic Compute Cloud (EC2). EC2 provides a server that has the capability to run continuously on the cloud. It also is cost-effective, scalable, and allows for remote server access.

Creating the EC2 Instance

The first step is to launch the instance that would host the backend API.

In the AWS Management Console:

1. Navigate to EC2 Dashboard
2. Select the Instances tab in the left side bar
3. Click Launch Instance
4. Select operating system image
 - a. Choose Ubuntu
 - i. Stable Linux environment
 - ii. Extensive package support
5. Select instance type
 - a. T3.micro
6. Create SSH-Pair key
 - a. This key is used to create a secure connection to the server. Connection to the server is not possible without this key.
7. Configure the security groups by adding rules of what ports are accessible

Port	Protocol	Purpose
22	TCP	SSH Access
8000	TCP	FastAPI server

Port 8000 allows for the Unity connection

Connecting to the Server

Connection to the server is done using SSH (Secure Shell)

To connect to the server the user must have

- EC2 Public IP address
 - Example: 18.217.36.198
- The SSH key pair (.pem file)
- Terminal application to get access

Connecting on Mac

1. Open terminal
2. Locate the private key (often found in downloads)
 - a. **Example key:** Key.pem
 - b. **Example location:** ~/Downloads/Key.pem
3. Navigate to the key directory
 - a. `cd ~/Downloads`
4. Set proper key permissions, SSH requires private keys have certain security permissions
 - a. `chmod 400 IXL-Key.pem`
5. Connect to the server by SSH command
 - a. `ssh -i Key.pem ubuntu@18.217.36.198`
6. The first time you connect will ask if you want to connect, type **yes**
7. If connected properly, the prompt will change to something like:
 - a. **ubuntu@ip-172-31-xx-xx:~\$**

Connecting from Windows

Recommend using PowerShell with OpenSSH

1. Open PowerShell
2. Go to the key file
 - a. `cd C:\Users\YourUsername\Downloads`
3. Connect using SSH
 - a. `ssh -i IXL-Key.pem ubuntu@18.217.36.198`
4. The first time you connect will ask if you want to connect, type **yes**
5. If connected properly, the prompt will show something like this:
 - a. Welcome to Ubuntu 22.04 LTS
ubuntu@ip-172-31-xx-xx:~\$

Securely Storing the OpenAI API Key Using AWS Parameter Store

To protect the OpenAI API key, the key was stored as a parameter and not directly in the code.

1. Open AWS Systems Manager in the AWS console
2. Navigate to Parameter Store
3. Click Create Parameter
4. Fill in the following information
 - a. **Name:** /openai/api_key
 - b. **Type:** SecureString
 - c. **Data Type:** text
 - d. **Value:** OpenAI API key goes here

Creating an IAM Role for the EC2 Instance

1. Open AWS IAM Console
2. Select Roles
3. Click on Create Role
4. Creating the Role
 - a. Choose
 - i. Trusted entity: AWS Service
 - Use case: EC2
 - b. Attach the following policy:
 - i. AmazonSSMReadOnlyAccess
 - c. Name the role
 - i. EC2-ParameterStore-Access

This allows for the EC2 server to read parameters stored in Systems Manager. Now we have to attach the IAM Role to the Instance.

1. Open AWS EC2 Dashboard
2. Select the running instance
3. Click on Actions in the top left
4. Select Security and then Modify IAM Role
5. Choose the role:
 - a. EC2-ParameterStore-Access
6. Click Update IAM Role

The EC2 Server can now access the stored API key. Now we have to use it in the EC2 server.

1. Within the server run
 - a. `pip install boto3`
 - b.

Once you create the [server.py](#) and implement the API key you can verify access using these steps.

1. Run the Python interpreter
 - a. python3
2. Run this code

```
import boto3
```

```
ssm = boto3.client("ssm", region_name="us-east-2")
```

```
print(ssm.get_parameter(Name="/openai/api_key", WithDecryption=True))
```

If configured correctly, the command will return the stored API key.

Preparing the Server Environment

1. Update system packages
 - a. sudo apt update
2. Install Python package manager
 - a. sudo apt install python3-pip -y
3. Install Python dependencies
 - a. pip install fastapi uvicorn openai boto3 pydantic

Uploading Project Files

The knowledge base is built off of pdf's full of research and interview content which have to be uploaded to the server.

On Mac

1. Exit the SSH terminal
 - a. exit
2. Navigate to folder containing the PDF files
 - a. cd ~/Downloads/RAG-PDF-Uploads
3. To confirm they are present, run:
 - a. ls
 - b. Should see something like:
Document1.pdf
Document2.pdf
Document3.pdf
4. Run the following command to upload the entire folder to the server
 - a. scp -i ~/Downloads/Key.pem -r ~/Downloads/RAG-PDF-Uploads ubuntu@18.217.36.198:/home/ubuntu/

5. Verify upload of documents to the server
 - a. Reconnect to SSH
 - i. `ssh -i ~/Downloads/Key.pem ubuntu@18.217.36.198`
 - b. Check the contents of the server
 - i. `ls`
 - c. Should see a list of documents including the folder you just uploaded.
 - d. Check contents of the uploaded folder
 - i. `ls RAG-PDF-Uploads`
 - e. Should see the list of documents within the folder

On Windows

1. Exit out of SSH terminal
 - a. `exit`
2. Navigate to folder containing the PDF files
 - a. `cd C:\Users\YourUsername\Downloads\RAG-PDF-Uploads`
3. To confirm they are present, run:
 - a. `dir`
 - b. Should see something like:
Document1.pdf
Document2.pdf
Document3.pdf
4. Run the following command to upload the entire folder to the server
 - a. `scp -i C:\Users\YourUsername\Downloads\Key.pem -r C:\Users\YourUsername\Downloads\RAG-PDF-Uploads ubuntu@18.217.36.198:/home/ubuntu/`
5. Verify upload of documents to the server
 - a. Reconnect to SSH
 - i. `ssh -i C:\Users\YourUsername\Downloads\Key.pem ubuntu@18.217.36.198`
 - b. Check the contents of the server
 - i. `ls`
 - c. Should see a list of documents including the folder you just uploaded.
 - d. Check contents of the uploaded folder
 - i. `ls RAG-PDF-Uploads`
 - e. Should see the list of documents within the folder

Creating the Project Directory

Make sure you are SSH into the server.

1. Once on the server, run these commands
 - a. `mkdir rag-server`
`cd rag-server`
2. Verify location

- a. `pwd`

Creating the Vector Store and Getting the Vector Store ID

1. Make sure you are in the project directory in the EC2 server
 - a. `cd ~/rag-server`
2. Start the Python interpreter
 - a. `python3`
3. Run the following code inside the Python interpreter

```
from openai import OpenAI
```

```
client = OpenAI()
```

```
vector_store = client.vector_stores.create(  
    name="rag-knowledge-base"  
)
```

```
print(vector_store.id)
```

4. The output is the vector store ID needed in [server.py](#) and `upload_pdfs.py`
5. Exit Python
 - a. `exit()`

Creating the Backend API File ([server.py](#))

1. Create the file
 - a. `nano server.py`
2. This is the code in the [server.py](#) document

```
import boto3  
from fastapi import FastAPI, HTTPException  
from pydantic import BaseModel  
from openai import OpenAI  
import logging
```

```
# -----  
# App setup
```

```

# -----
app = FastAPI(title="IXL AI RAG Server")
logging.basicConfig(level=logging.INFO)

# -----
# Secure OpenAI key retrieval
# -----
def get_openai_key() -> str:
    try:
        ssm = boto3.client("ssm", region_name="us-east-2")
        response = ssm.get_parameter(
            Name="/openai/api_key",
            WithDecryption=True
        )
        return response["Parameter"]["Value"]
    except Exception as e:
        logging.error("Failed to retrieve OpenAI API key")
        raise RuntimeError("OpenAI API key not available") from e
# Initialize OpenAI client ONCE
client = OpenAI(api_key=get_openai_key())

# -----
# Vector Store
# -----
VECTOR_STORE_ID = "Vector Store ID Here"

# -----
# Request / Response Models
# -----
class Question(BaseModel):
    question: str
class Answer(BaseModel):
    answer: str

# -----
# Health check (browser test)
# -----
@app.get("/")
def root():
    return {
        "status": "running",
        "service": "IXL AI RAG Server"
    }

```



```

# -----
# Unity → AI endpoint
# -----
@app.post("/ask", response_model=Answer)
def ask_ai(data: Question):
    if not data.question.strip():
        raise HTTPException(status_code=400, detail="Question cannot be empty")

    try:
        response = client.responses.create(
            model="gpt-4.1-mini",
            input=data.question,
            tools=[
                {
                    "type": "file_search",
                    "vector_store_ids": [VECTOR_STORE_ID]
                }
            ],
        )

        return Answer(answer=response.output_text)

    except Exception as e:
        logging.exception("OpenAI request failed")
        raise HTTPException(
            status_code=500,
            detail="AI service failed to process the request"
        )

```

3. Save and exit out of the file.

- a. CTRL + O
- b. ENTER
- c. CTRL + X

Creating the Upload Script ([upload_pdfs.py](#))

This script is what uploads PDFs to the vector store

1. Create the file
 - a. nano [upload_pdfs.py](#)
2. Code inside of upload_pdfs.py

```

from openai import OpenAI
import os

client = OpenAI()

VECTOR_STORE_ID = "VECTOR_STORE_ID_HERE" # <-- replace this
PDF_DIR = "knowledge"

file_ids = []

for filename in os.listdir(PDF_DIR):
    if filename.endswith(".pdf"):
        path = os.path.join(PDF_DIR, filename)
        with open(path, "rb") as f:
            uploaded = client.files.create(
                file=f,
                purpose="assistants"
            )
            file_ids.append(uploaded.id)
            print(f"Uploaded {filename}")

client.vector_stores.file_batches.create(
    vector_store_id=VECTOR_STORE_ID,
    file_ids=file_ids
)

print("All PDFs added to vector store.")

```

3. Move the PDFs to the Knowledge folder
 - a. `mv RAG-PDF-Uploads/* knowledge/`
4. Verify
 - a. `ls knowledge`

Uploading PDFs to the Vector Store

1. Run the upload script
 - a. `python3 upload_pdfs.py`
2. Output:
 - a. All PDFs added to vector store.

Running and Testing the FastAPI Server

1. Start the API manually:
 - a. `uvicorn server:app --host 0.0.0.0 --port 8000`
2. Open a browser and visit:
 - a. <http://18.217.36.198:8000/docs>
3. Test the endpoint in POST /ask

Running the Server 24/7

1. Create the service file
 - a. `sudo nano /etc/systemd/system/rag-api.service`
2. This is the code that should go within the file

[Unit]

Description=RAG FastAPI Server

After=network.target

[Service]

User=ubuntu

WorkingDirectory=/home/ubuntu/rag-server

ExecStart=/usr/bin/python3 -m uvicorn server:app --host 0.0.0.0 --port 8000

Restart=always

[Install]

WantedBy=multi-user.target

3. Save and Exit
4. Reload services
 - a. `sudo systemctl daemon-reload`
5. Start server
 - a. `sudo systemctl start rag-api`
6. Enable Auto Start
 - a. `sudo systemctl enable rag-api`
7. Check status
 - a. `sudo systemctl status rag-api`

Giving Another User Service Access

1. Add user to sudo group

- a. `sudo usermod -aG sudo USERNAME`
- 2. Set Password:
 - a. `sudo passwd Password`

Conclusion

This system creates a backend RAG server connected to our OpenAI API to give answers to questions asked by users through Unity. This ensures a secure build by protecting the keys needed to access these servers while also allowing multiple users access to the server to work on enhancing the knowledge base.